핵심 인터페이스 및 데이터 구조

개요

이 문서는 arkworks-rs/poly-commit 라이브러리의 핵심 인터페이스와 데이터 구조를 상세히 분석합니다. 다항식 커밋먼트 스킴의 추상화 계층과 구체적인 구현에서 사용되는 주요 타입들을 이해할 수 있도록 구성되었습니다.

PolynomialCommitment 트레이트

트레이트 정의

`PolynomialCommitment` 트레이트는 모든 다항식 커밋먼트 스킴이 구현해야 하는 핵심 인터페이스입니다:

```
pub trait PolynomialCommitment<F: PrimeField, P: Polynomial<F>>: Sized {
// 연관 타입들
type UniversalParams: PCUniversalParams;
type CommitterKey: PCCommitterKey;
type VerifierKey: PCVerifierKey;
type Commitment: PCCommitment + Default;
type CommitmentState: PCCommitmentState;
type Proof: Clone;
type BatchProof: Clone + From<Vec<Self::Proof>> + Into<Vec<Self::Proof>>;
type Error: ark_std::error::Error + From<Error>;
```

```rust
// 핵심 메서드들
fn setup<R: RngCore>(
max_degree: usize,
num_vars: Option<usize>,
rng: &mut R,
) -> Result<Self::UniversalParams, Self::Error>;

fn trim(
pp: &Self::UniversalParams,
supported_degree: usize,
supported_hiding_bound: usize,
enforced_degree_bounds: Option<&[usize]>,
) -> Result<(Self::CommitterKey, Self::VerifierKey), Self::Error>;

fn commit<'a>(
ck: &Self::CommitterKey,
polynomials: impl IntoIterator<Item = &'a LabeledPolynomial<F, P>>,
rng: Option<&mut dyn RngCore>,
) -> Result<(Vec<LabeledCommitment<Self::Commitment>>, Vec<Self::CommitmentState
where P: 'a;

fn open<'a>(/* ... */) -> Result<Self::Proof, Self::Error>;
```

```
fn check<'a>(/* ... */) -> Result<bool, Self::Error>;

fn batch_open<'a>(/* ... */) -> Result<Self::BatchProof, Self::Error>;

fn batch_check<'a, R: RngCore>(/* ... */) -> Result<bool, Self::Error>;

fn open_combinations<'a>(/* ... */) -> Result<BatchLCProof<F, Self::BatchProof>,

fn check_combinations<'a, R: RngCore>(/* ... */) -> Result<bool, Self::Error>;

}
```

연관 타입들의 역할

UniversalParams

- **목적**: 스킴의 전역 공개 매개변수를 나타냅니다
- **특징**: `PCUniversalParams` 트레이트를 구현하며, 최대 지원 차수 정보를 제공합니다
- **KZG10 예시**: trusted setup에서 생성된 powers of G와 관련 매개변수들을 포함합니다

CommitterKey와 VerifierKey

- **CommitterKey**: 다항식 커밋과 증명 생성에 필요한 키
- **VerifierKey**: 증명 검증에 필요한 키
- **관계**: `trim` 메서드를 통해 `UniversalParams` 에서 파생됩니다
- **특징**: 각각 `PCCommitterKey` , `PCVerifierKey` 트레이트를 구현합니다

Commitment

- **목적**: 다항식에 대한 커밋먼트를 나타냅니다
- **특징**: `PCCommitment` 트레이트를 구현하며, `Default` 트레이트도 구현합니다
- **KZG10 예시**: 타원곡선 그룹의 원소로 표현됩니다

CommitmentState

- **목적**: 커밋 과정에서 생성되는 보조 상태 정보를 저장합니다
- **용도**: `open` 단계에서 재사용되는 정보 (예: 커밋 랜덤성)
- **보안**: 검증자와 공유되지 않는 커미터 전용 정보입니다

Proof와 BatchProof

- **Proof**: 단일 점에서의 평가 증명
- **BatchProof**: 여러 점에서의 배치 평가 증명
- **변환**: `BatchProof` 는 `Vec<Proof>` 와 상호 변환 가능합니다

핵심 메서드들의 역할

setup

```
fn setup<R: RngCore>(
max_degree: usize,
num_vars: Option<usize>,
rng: &mut R,
) -> Result<Self::UniversalParams, Self::Error>;
```

- **목적**: 스킴의 전역 매개변수를 생성합니다
- **매개변수**:
- `max_degree` : 지원할 최대 다항식 차수
- `num_vars` : 다변수 다항식의 경우 변수 개수
- `rng` : 안전한 랜덤성 생성기
- **보안**: trusted setup이 필요한 스킴에서는 이 단계가 매우 중요합니다

trim

```
fn trim(
pp: &Self::UniversalParams,
supported_degree: usize,
supported_hiding_bound: usize,
enforced_degree_bounds: Option<&[usize]>,
) -> Result<(Self::CommitterKey, Self::VerifierKey), Self::Error>;
```

- **목적**: 전역 매개변수를 특정 용도에 맞게 특화합니다
- **최적화**: 실제 필요한 차수에 맞춰 키 크기를 줄입니다
- **차수 제한**: `enforced_degree_bounds` 로 엄격한 차수 제한을 설정할 수 있습니다

commit

```
fn commit<'a>(
ck: &Self::CommitterKey,
polynomials: impl IntoIterator<Item = &'a LabeledPolynomial<F, P>>,
rng: Option<&mut dyn RngCore>,
) -> Result<(Vec<LabeledCommitment<Self::Commitment>>, Vec<Self::CommitmentState
```

- **목적**: 다항식들에 대한 커밋먼트를 생성합니다
- **배치 처리**: 여러 다항식을 동시에 커밋할 수 있습니다
- **hiding**: `rng` 가 제공되면 hiding 커밋먼트를 생성합니다
- **출력**: 커밋먼트와 상태 정보를 함께 반환합니다

open과 check

- **open**: 특정 점에서의 다항식 평가값에 대한 증명을 생성합니다
- **check**: 생성된 증명의 유효성을 검증합니다
- **상호 작용**: open에서 생성된 증명은 check에서 검증됩니다

배치 연산들

- **batch_open/batch_check**: 여러 다항식을 여러 점에서 동시에 처리합니다
- **성능 최적화**: 개별 연산보다 효율적입니다
- **기본 구현**: 트레이트에서 기본 구현을 제공하지만, 각 스킴에서 최적화된 구현을 제공할 수 있습니다

핵심 데이터 구조

LabeledPolynomial

```
pub struct LabeledPolynomial<F: Field, P: Polynomial<F>> {
label: PolynomialLabel, // 다항식 식별자
polynomial: P, // 실제 다항식
degree_bound: Option<usize>, // 차수 제한 (선택적)
hiding_bound: Option<usize>, // hiding 쿼리 수 제한 (선택적)
_field: PhantomData<F>,
}
```

주요 특징

- **라벨링**: 각 다항식을 문자열로 식별합니다

- **차수 제한**: `degree_bound` 로 다항식의 최대 차수를 제한할 수 있습니다
- **hiding 지원**: `hiding_bound` 가 설정되면 hiding 커밋먼트를 생성합니다
- **제네릭**: 다양한 필드와 다항식 타입을 지원합니다

주요 메서드

```
impl<F: Field, P: Polynomial<F>> LabeledPolynomial<F, P> {
pub fn new(
label: PolynomialLabel,
polynomial: P,
degree_bound: Option<usize>,
hiding_bound: Option<usize>,
) -> Self;

pub fn label(&self) -> &String;
pub fn polynomial(&self) -> &P;
pub fn evaluate(&self, point: &P::Point) -> F;
pub fn degree(&self) -> usize;
pub fn degree_bound(&self) -> Option<usize>;
pub fn is_hiding(&self) -> bool;
pub fn hiding_bound(&self) -> Option<usize>;
}
```

LabeledCommitment

```
pub struct LabeledCommitment<C: PCCommitment> {
label: PolynomialLabel, // 다항식 식별자
commitment: C, // 실제 커밋먼트
degree_bound: Option<usize>, // 차수 제한 정보
}
```

특징

- **연결성**: `LabeledPolynomial` 과 동일한 라벨을 사용합니다
- **차수 정보**: 커밋먼트에 차수 제한 정보를 포함합니다
- **제네릭**: 다양한 커밋먼트 타입을 지원합니다

LinearCombination

```
pub struct LinearCombination<F> {
pub label: String, // 선형결합 식별자
pub terms: Vec<(F, LCTerm)>, // (계수, 항) 쌍들
}

pub enum LCTerm {
One, // 상수항 1
PolyLabel(String), // 다항식 라벨
}
```

용도

- **복잡한 쿼리**: 여러 다항식의 선형결합에 대한 증명을 효율적으로 처리합니다
- **R1CS 지원**: 제약 시스템에서 선형 제약을 표현합니다
- **배치 최적화**: 관련된 다항식들을 함께 처리하여 성능을 향상시킵니다

연산 지원

```
impl<F: Field> LinearCombination<F> {
pub fn empty(label: impl Into<String>) -> Self;
pub fn new(label: impl Into<String>, terms: Vec<(F, impl Into<LCTerm>)>) -> Self
pub fn push(&mut self, term: (F, LCTerm)) -> &mut Self;
}

// 산술 연산 지원
impl<F: Field> AddAssign<(F, &LinearCombination<F>)> for LinearCombination<F>;
impl<F: Field> SubAssign<(F, &LinearCombination<F>)> for LinearCombination<F>;
impl<F: Field> MulAssign<F> for LinearCombination<F>;
```

타입 별칭들

```
pub type QuerySet<T> = BTreeSet<(String, (String, T))>;
pub type Evaluations<T, F> = BTreeMap<(String, T), F>;
pub type PolynomialLabel = String;
```

QuerySet

- **구조**: `(다항식_라벨, (점_라벨, 점))` 형태의 집합
- **용도**: 어떤 다항식을 어떤 점에서 평가할지 지정합니다
- **예시**: `("poly1", ("alpha", field_element))`

Evaluations

- **구조**: `(다항식_라벨, 점) -> 평가값` 매핑
- **용도**: 쿼리 결과를 저장합니다
- **검증**: 증명 검증 시 주장된 평가값과 비교됩니다

트레이트 계층구조

PCUniversalParams

```
pub trait PCUniversalParams: Clone + Debug + CanonicalSerialize + CanonicalDeser
fn max_degree(&self) -> usize;
}
```

PCCommitterKey / PCVerifierKey

```
pub trait PCCommitterKey: Clone + Debug + CanonicalSerialize + CanonicalDeserial
fn max_degree(&self) -> usize;
fn supported_degree(&self) -> usize;
}
```

```
pub trait PCVerifierKey: Clone + Debug + CanonicalSerialize + CanonicalDeseriali

fn max_degree(&self) -> usize;

fn supported_degree(&self) -> usize;

}
```

PCCommitment

```
pub trait PCCommitment: Clone + CanonicalSerialize + CanonicalDeserialize {

fn empty() -> Self;

fn has_degree_bound(&self) -> bool;

}
```

PCCommitmentState

```
pub trait PCCommitmentState: Clone + CanonicalSerialize + CanonicalDeserialize {

type Randomness: Clone + CanonicalSerialize + CanonicalDeserialize;

fn empty() -> Self;

fn rand<R: RngCore>(

num_queries: usize,

has_degree_bound: bool,

num_vars: Option<usize>,

rng: &mut R,

) -> Self::Randomness;
```

```
}
```

KZG10 구체적 구현 예시

데이터 구조들

```
// KZG10의 UniversalParams
pub struct UniversalParams<E: Pairing> {
pub powers_of_g: Vec<E::G1Affine>, // [G, βG, β²G, ...]
pub powers_of_gamma_g: BTreeMap<usize, E::G1Affine>, // [γG, βγG, β²γG, ...]
pub h: E::G2Affine, // G2 생성원
pub beta_h: E::G2Affine, // βH
pub neg_powers_of_h: BTreeMap<usize, E::G2Affine>, // [H, β⁻¹H, β⁻²H, ...]
pub prepared_h: E::G2Prepared, // 페어링 최적화용
pub prepared_beta_h: E::G2Prepared, // 페어링 최적화용
}

// KZG10의 Commitment
pub struct Commitment<E: Pairing>(pub E::G1Affine);

// KZG10의 Proof
pub struct Proof<E: Pairing> {
pub w: E::G1Affine, // witness polynomial 커밋먼트
```

```
pub random_v: Option<E::ScalarField>, // hiding을 위한 랜덤 평가값
}

// KZG10의 Randomness (CommitmentState)
pub struct Randomness<F: PrimeField, P: DenseUVPolynomial<F>> {
pub blinding_polynomial: P, // hiding을 위한 랜덤 다항식
_field: PhantomData<F>,
}
```

트레이트 구현 예시

```
impl<E, P> PolynomialCommitment<E::ScalarField, P> for KZG10<E, P>
where
E: Pairing,
P: DenseUVPolynomial<E::ScalarField, Point = E::ScalarField>,
{
type UniversalParams = UniversalParams<E>;
type CommitterKey = Powers<E>;
type VerifierKey = VerifierKey<E>;
type Commitment = Commitment<E>;
type CommitmentState = Randomness<E::ScalarField, P>;
type Proof = Proof<E>;
type BatchProof = Vec<Proof<E>>;
```

```
type Error = Error;



// 메서드 구현들...

}
```

에러 처리

Error enum 분석

라이브러리는 포괄적인 에러 처리를 제공합니다:

```
pub enum Error {

// 다항식 관련 에러

MissingPolynomial { label: String },

MissingEvaluation { label: String },

MissingLHS { label: String },

// 설정 관련 에러

MissingRng,

DegreeIsZero,

TooManyCoefficients { num_coefficients: usize, num_powers: usize },

// Hiding 관련 에러

HidingBoundIsZero,

HidingBoundToolarge { hiding_poly_degree: usize, num_powers: usize },

// 차수 제한 관련 에러
```

```
TrimmingDegreeTooLarge,

EmptyDegreeBounds,

EquationHasDegreeBounds(String),

UnsupportedDegreeBound(usize),

IncorrectDegreeBound {

poly_degree: usize,

degree_bound: usize,

supported_degree: usize,

label: String

},

// 입력 검증 에러

IncorrectInputLength(String),

InvalidNumberOfVariables,

PolynomialDegreeTooLarge {

poly_degree: usize,

supported_degree: usize,

label: String

},

// 검증 관련 에러

InvalidCommitment,

IncorrectCommitmentSize { encountered: usize, expected: usize },

TranscriptError,

// 스킴별 특수 에러
```

```
InvalidParameters(String),

HashingError,

EncodingError,

// 라벨 불일치 에러

MismatchedLabels {

commitment_label: String,

polynomial_label: String

},

MismatchedNumVars { poly_nv: usize, point_nv: usize },

}
```

에러 발생 조건과 대응 방법

설정 단계 에러

- **DegreeIsZero**: 차수 0 다항식은 지원되지 않습니다
- **InvalidNumberOfVariables**: 다변수 설정에서 잘못된 변수 개수가 제공되었습니다
- **대응**: setup 호출 시 적절한 매개변수를 제공해야 합니다

커밋 단계 에러

- **TooManyCoefficients**: 다항식 차수가 지원 범위를 초과했습니다
- **MissingRng**: hiding 커밋먼트에 필요한 RNG가 제공되지 않았습니다
- **HidingBoundIsZero/HidingBoundToolarge**: hiding bound 설정이 잘못되었습니다
- **대응**: 적절한 차수와 hiding 매개변수를 사용해야 합니다

증명 생성/검증 에러

- **MissingPolynomial/MissingEvaluation**: 쿼리셋에 참조된 다항식이나 평가값이 없습니다
- **InvalidCommitment**: 커밋먼트 검증에 실패했습니다
- **MismatchedLabels**: 커밋먼트와 다항식의 라벨이 일치하지 않습니다
- **대응**: 일관된 라벨링과 완전한 데이터를 제공해야 합니다

차수 제한 에러

- **UnsupportedDegreeBound**: 요청된 차수 제한이 지원되지 않습니다
- **IncorrectDegreeBound**: 차수 제한이 다항식 차수나 지원 범위와 맞지 않습니다
- **대응**: trim 단계에서 적절한 차수 제한을 설정해야 합니다

에러 처리 모범 사례

```
// 1. 설정 단계에서의 에러 처리
match KZG10::setup(max_degree, false, rng) {
Ok(params) => params,
Err(Error::DegreeIsZero) => {
// 최소 차수 1 이상으로 재시도
return KZG10::setup(1, false, rng);
},
Err(e) => return Err(e),
}

// 2. 커밋 단계에서의 에러 처리
match KZG10::commit(&ck, &polynomials, Some(rng)) {
```

```
Ok((commitments, states)) => (commitments, states),

Err(Error::TooManyCoefficients { num_coefficients, num_powers }) => {

eprintln!("다항식 차수({})가 지원 범위({})를 초과했습니다",

num_coefficients - 1, num_powers - 1);

return Err(Error::PolynomialDegreeTooLarge {

poly_degree: num_coefficients - 1,

supported_degree: num_powers - 1,

label: "unknown".to_string()

});

},

Err(e) => return Err(e),

}



// 3. 검증 단계에서의 에러 처리

match KZG10::check(&vk, &commitment, point, value, &proof) {

Ok(true) => println!("증명이 유효합니다"),

Ok(false) => println!("증명이 무효합니다"),

Err(Error::InvalidCommitment) => {

eprintln!("커밋먼트 형식이 올바르지 않습니다");

},

Err(e) => eprintln!("검증 중 에러 발생: {}", e),

}
```

사용 패턴과 모범 사례

기본 사용 패턴

```rust
use ark_poly_commit::{PolynomialCommitment, LabeledPolynomial};
use ark_poly::DenseUVPolynomial;

// 1. 설정
let max_degree = 100;
let pp = PCS::setup(max_degree, None, rng)?;
let (ck, vk) = PCS::trim(&pp, degree, hiding_bound, Some(&degree_bounds))?;

// 2. 다항식 준비
let poly = DensePolynomial::rand(degree, rng);
let labeled_poly = LabeledPolynomial::new("my_poly", poly, None, Some(1));

// 3. 커밋
let (commitments, states) = PCS::commit(&ck, &[&labeled_poly], Some(rng))?;

// 4. 증명 생성
let point = Field::rand(rng);
let proof = PCS::open(&ck, &[&labeled_poly], &commitments, &point,
&mut sponge, &states, Some(rng))?;
```

```
// 5. 검증
let value = labeled_poly.evaluate(&point);
let result = PCS::check(&vk, &commitments, &point, &[value],
&proof, &mut sponge, Some(rng))?;
```

배치 처리 패턴

```
// 여러 다항식을 여러 점에서 동시에 처리
let mut query_set = QuerySet::new();
let mut evaluations = Evaluations::new();

for (i, poly) in polynomials.iter().enumerate() {
let point = points[i];
let label = format!("poly_{}", i);
let point_label = format!("point_{}", i);
query_set.insert((label.clone(), (point_label, point)));
evaluations.insert((label, point), poly.evaluate(&point));
}

let batch_proof = PCS::batch_open(&ck, &polynomials, &commitments,
&query_set, &mut sponge, &states, Some(rng))?;
```

```
let result = PCS::batch_check(&vk, &commitments, &query_set, &evaluations,

&batch_proof, &mut sponge, rng)?;
```

이 문서를 통해 poly-commit 라이브러리의 핵심 추상화와 구체적인 구현을 이해할 수 있습니다.

관련 문서

이전 단계

- **프로젝트 개요**: 라이브러리 전체 구조와 지원 스킴들의 개요

다음 단계

- **KZG10 구현 분석**: 이 인터페이스를 구현한 KZG10 스킴의 구체적 구현
- **Marlin PC 구현 분석**: 고급 기능을 추가한 Marlin PC의 구현 방식

실습 및 참고

- **사용 예제 및 실습 가이드**: 이 인터페이스들을 실제로 사용하는 방법
- **암호학적 배경**: 각 트레이트와 데이터 구조의 암호학적 의미
- **용어집**: 관련 용어와 개념 정의